

Java Backend Architecture

Interview Series

Chapter 11.3

ETags & HTTP Conditional Requests

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

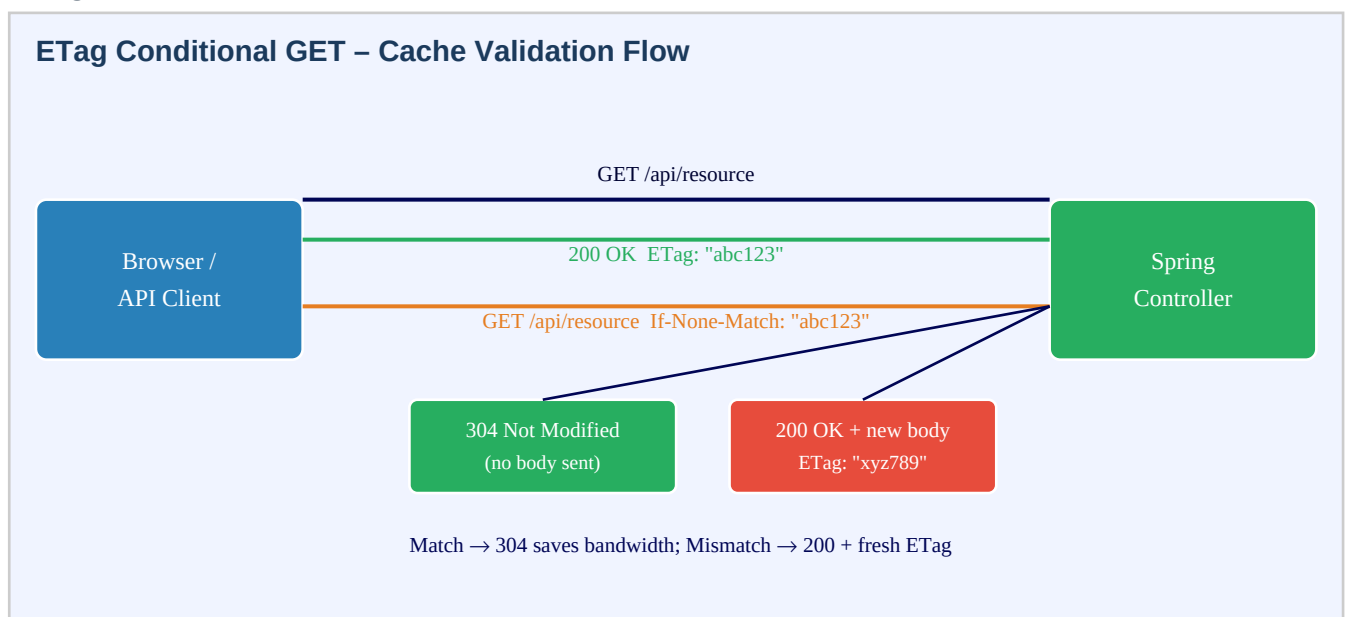
Chapter 11.3 – ETags & HTTP Conditional Requests

Introduction

An ETag (Entity Tag) is an HTTP response header that carries an opaque identifier — typically a hash (MD5/SHA) of the response body or a version number — uniquely identifying a specific version of a resource. ETags are the foundation of HTTP conditional requests defined in RFC 7232. They allow clients to make GET requests conditional ("only return the body if it changed") and PUT/PATCH/DELETE requests conditional ("only apply if no one else has modified the resource"). This dual role makes ETags valuable both for bandwidth optimization and for safe concurrent writes.

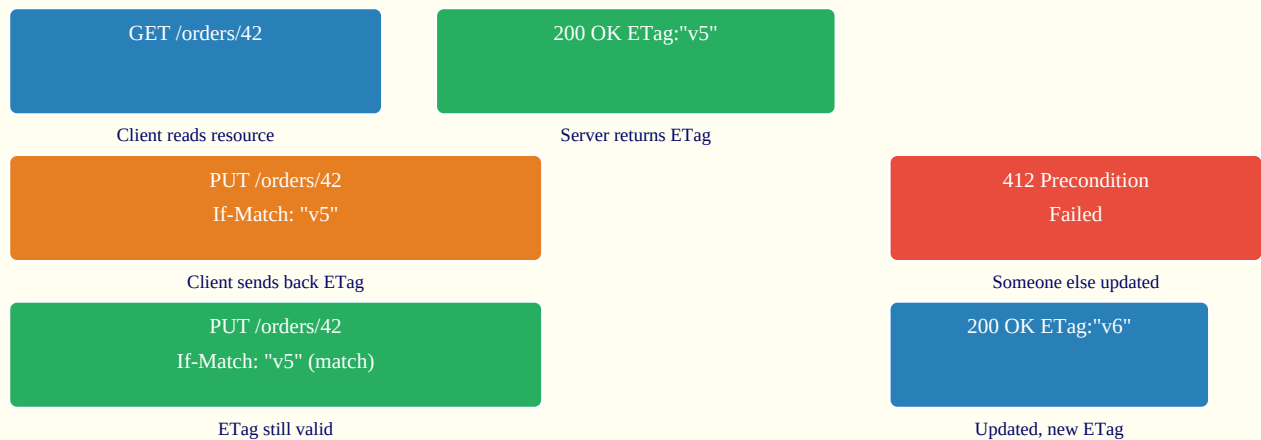
In Java backends, Spring MVC provides `ShallowEtagHeaderFilter` for automatic ETag generation from the serialized response body, and the `WebRequest/WebResponse` API for deep ETag checks before executing expensive database queries. Combined with `Cache-Control` headers, ETags form the backbone of HTTP caching infrastructure, reducing server load and improving client performance for both browser applications and API consumers.

ETag Conditional GET Flow



If-Match on PUT/PATCH – Optimistic Gate

If-Match on PUT/PATCH – Optimistic Update Gate



Shallow vs Deep ETag in Spring

Shallow ETag vs Deep ETag in Spring



Shallow: easy setup via filter | Deep: manual but full optimization

Key Definitions

ETag	HTTP response header with an opaque validator (hash or version) identifying a resource version
If-None-Match	Request header on GET: "respond 200 only if ETag changed; else 304"
If-Match	Request header on PUT/PATCH/DELETE: "apply only if current ETag matches; else 412"
304 Not Modified	Server response when If-None-Match matches: headers only, no body; client uses cached copy
412 Precondition Failed	Server response when If-Match does not match: write rejected, resource was modified
Strong ETag	ETag representing byte-for-byte identical response (default); comparison is exact
Weak ETag	ETag prefixed W/: semantically equivalent but not byte-identical; suitable for gzip variants

ShallowEtagHeaderFilter	Spring Servlet filter that generates ETag from MD5 of response body bytes automatically
Deep ETag	Application-level ETag computed from entity version/hash, checked before DB query
Cache-Control	HTTP header controlling caching policy (max-age, no-cache, private, public, must-revalidate)
Conditional GET	GET with If-None-Match or If-Modified-Since; enables bandwidth-saving 304 responses
RFC 7232	HTTP/1.1 Conditional Requests specification defining ETag semantics and If-* headers
Last-Modified	Response header with timestamp; paired with If-Modified-Since for time-based validation
Optimistic locking (HTTP)	Using If-Match ETag to implement HTTP-layer optimistic concurrency for safe PUT/PATCH
Vary header	Indicates which request headers were used to generate the response; affects cache keying

ETag Types & Strategies Comparison

Aspect	Shallow ETag	Deep ETag	Last-Modified
Generation	MD5(response bytes)	Entity version/hash	Timestamp from DB
DB query saved?	No – always queries	Yes – check before query	Sometimes
Spring support	ShallowEtagHeaderFilter	WebRequest.checkNotModified()	WebRequest.checkNotModified(ts)
Best for	Easy setup, static resources	High-traffic APIs	File serving
Precision	Exact (strong)	Exact (strong)	Second granularity
Bandwidth saved	Yes (no body on 304)	Yes (no body on 304)	Yes (no body on 304)

Code Examples

1. ShallowEtagHeaderFilter – auto ETag generation

```
// Register ShallowEtagHeaderFilter in Spring Boot @Configuration public class WebConfig {
@Bean public FilterRegistrationBean<ShallowEtagHeaderFilter> shallowEtagBean() {
    FilterRegistrationBean<ShallowEtagHeaderFilter> frb = new FilterRegistrationBean<>(new
    ShallowEtagHeaderFilter()); frb.addUrlPatterns("/api/*"); frb.setName("etagFilter"); return
    frb; } } // ShallowEtagHeaderFilter: // 1. Buffers the full response body // 2. Computes MD5
    hash → ETag: "686897696a7c876b7e" // 3. If request has If-None-Match matching computed ETag →
    returns 304 (no body) // 4. Otherwise → forwards 200 with body and ETag header // // CAVEAT:
    DB is still queried every request (body is computed to get the hash) // Use for low-traffic
    endpoints or as a quick baseline
```

2. Deep ETag with WebRequest – skip DB on match

```
@RestController @RequestMapping("/api/products") public class ProductController { private
final ProductRepository repo; @GetMapping("/{id}") public ResponseEntity<Product> getProduct(
@PathVariable String id, WebRequest webRequest) { // Fetch only the version first (cheap query)
Long version = repo.findVersionById(id) .orElseThrow(() -> new ResourceNotFoundException(id));
// Build ETag from version – no need to load full entity yet String etag = '' +
version.toString() + ''; // checkNotModified: compares etag against If-None-Match header // If
match → sets 304 status and returns true → return null body if
(webRequest.checkNotModified(etag)) { return null; // Spring returns 304 automatically } //
ETag did not match → load full entity and return 200 Product product = repo.findById(id)
.orElseThrow(() -> new ResourceNotFoundException(id)); return ResponseEntity.ok() .eTag(etag)
.cacheControl(CacheControl.maxAge(60, TimeUnit.SECONDS)) .body(product); } } // Repository
lightweight query: // @Query("SELECT p.version FROM Product p WHERE p.id = :id") //
Optional<Long> findVersionById(@Param("id") String id);
```

3. If-Match on PUT – reject stale writes with 412

```
@PutMapping("/{id}") public ResponseEntity<Product> updateProduct( @PathVariable String id,
@RequestHeader(value = "If-Match", required = false) String ifMatch, @RequestBody
ProductUpdateRequest req) { Product current = repo.findById(id) .orElseThrow(() -> new
ResourceNotFoundException(id)); // Validate If-Match header if (ifMatch != null) { String
currentEtag = '' + current.getVersion() + ''; // Strip quotes for comparison String
clientEtag = ifMatch.replace("\"", ""); if (!currentEtag.replace("\"", "").equals(clientEtag))
{ // 412 Precondition Failed – resource was modified by someone else return
ResponseEntity.status(HttpStatus.PRECONDITION_FAILED) .eTag(currentEtag) .body(current); //
return current state so client can reconcile } } // ETag matched (or no If-Match header) –
apply update current.setName(req.getName()); current.setPrice(req.getPrice()); Product saved =
repo.save(current); // @Version increments version String newEtag = '' + saved.getVersion() +
''; return ResponseEntity.ok() .eTag(newEtag) .body(saved); }
```

4. Cache-Control + ETag integration

```
@GetMapping("/catalog/{id}") public ResponseEntity<CatalogItem> getCatalogItem( @PathVariable
String id, WebRequest webRequest) { CatalogItem item = catalogService.findById(id); // Compute
ETag from content hash String etag = '' + DigestUtils.md5DigestAsHex(
objectMapper.writeValueAsBytes(item)) + ''; if (webRequest.checkNotModified(etag)) { return
null; } return ResponseEntity.ok() // Public cache (CDN + browser): serve cached copy for 5
minutes // After 5 min, must revalidate with server (sends If-None-Match)
.cacheControl(CacheControl.maxAge(5, TimeUnit.MINUTES) .cachePublic() .mustRevalidate())
.eTag(etag) .body(item); } // Response headers: // ETag: "d41d8cd98f00b204e9800998ecf8427e" //
Cache-Control: max-age=300, public, must-revalidate
```

5. ETag with database @Version entity

```
@Entity public class Order { @Id private String id; private String status; private BigDecimal
total; @Version private Long version; // used as ETag source } // ETag = version number →
deterministic, DB-native, no hashing needed // Client: GET /orders/42 → ETag: "7" // Client:
PUT /orders/42 → If-Match: "7" // Server: if order.getVersion() != 7 → 412 // if match →
update, new ETag: "8" // Advantage: version is already indexed, zero overhead // The @Version
column does double duty: OCC guard AND ETag source
```

6. Weak ETag for content negotiation (gzip variant)

```
// Strong ETags break when same content served compressed vs uncompressed // Spring
automatically weakens ETags when using ShallowEtagHeaderFilter + gzip // Strong: ETag: "abc123"
(exact bytes) // Weak: ETag: W/"abc123" (semantically equivalent) // If using gzip compression
```

```
+ ShallowEtagHeaderFilter, add: @Bean public FilterRegistrationBean<ShallowEtagHeaderFilter>
shallowEtag() { FilterRegistrationBean<ShallowEtagHeaderFilter> frb = new
FilterRegistrationBean<>(new ShallowEtagHeaderFilter()); frb.addUrlPatterns("/api/*"); //
Enable weak ETags to avoid mismatch after gzip encoding frb.setInitParameter("writeWeakETag",
"true"); return frb; } // RFC 7232: clients MUST use strong comparison for If-Match (PUT/DELETE
safety) // clients MAY use weak comparison for If-None-Match (caching)
```

7. ETag in REST client (RestTemplate / WebClient)

```
// Using WebClient to handle ETag caching @Component public class ProductApiClient { private
final WebClient webClient; private final Map<String, String> etagCache = new
ConcurrentHashMap<>(); private final Map<String, Object> responseCache = new
ConcurrentHashMap<>(); public Object getProduct(String id) { String cachedEtag =
etagCache.get(id); return webClient.get().uri("/products/{id}", id).headers(h -> { if
(cachedEtag != null) { h.setIfNoneMatch(cachedEtag); } }) .retrieve().toEntity(Object.class)
.map(response -> { if (response.getStatusCode() == HttpStatus.NOT_MODIFIED) { return
responseCache.get(id); // serve from local cache } // 200: update caches String newEtag =
response.getHeaders().getETag(); if (newEtag != null) etagCache.put(id, newEtag);
responseCache.put(id, response.getBody()); return response.getBody(); }) .block(); } }
```

Interview Questions & Answers

Q1. What is an ETag and what problem does it solve?

An ETag (Entity Tag) is an opaque string in the HTTP ETag response header that uniquely identifies a specific version of a resource. It solves two problems: (1) Bandwidth waste — without ETags, clients must re-download unchanged resources on every GET. With ETag + If-None-Match, the server returns 304 Not Modified (headers only, no body) if the resource hasn't changed, saving network bandwidth. (2) Lost update problem — without ETags on writes, two clients can both read a resource, both make changes, and the second write silently overwrites the first. With If-Match, the server rejects the write with 412 if someone else has modified the resource since the client read it.

Q2. What is the difference between If-None-Match and If-Match?

If-None-Match is used on GET/HEAD requests for cache validation: the client sends the ETag it has cached; if the server's current ETag matches (resource unchanged), the server returns 304 Not Modified with no body. If ETags differ, the server returns 200 with the updated body and new ETag. If-Match is used on state-changing requests (PUT/PATCH/DELETE) for safe concurrent writes: the client sends the ETag it based its modification on; if the server's current ETag matches (resource unchanged since the client read it), the write proceeds. If ETags differ (someone else modified the resource), the server returns 412 Precondition Failed and the write is rejected.

Q3. What is the difference between a 304 Not Modified and 412 Precondition Failed response?

304 Not Modified is a success response to a conditional GET. It tells the client: 'The resource hasn't changed since you last fetched it; use your cached copy.' The response body is empty, saving bandwidth. This is triggered by the If-None-Match header matching. 412 Precondition Failed is an error response to a conditional PUT/PATCH/DELETE. It tells the client: 'The precondition you specified (the ETag in If-Match) is no longer true; the resource was modified by someone else since you read it.' The write is rejected and the client must re-read the resource and decide how to reconcile the conflict.

Q4. What is ShallowEtagHeaderFilter and what are its limitations?

ShallowEtagHeaderFilter is a Spring Servlet filter that automatically generates ETags from the MD5 hash of the serialized response body bytes. It intercepts the response, buffers the body, computes MD5, sets the ETag header, and if the request's If-None-Match header matches, it discards the body and returns 304. Limitations: (1) The full response is still generated — the controller runs, the DB is queried, the body is serialized — all work is done before the ETag is checked. It saves bandwidth but not CPU or DB load. (2) Memory overhead from buffering the response. (3) Not suitable for high-traffic endpoints where the goal is to eliminate DB queries entirely.

Q5. What is a deep ETag and how does it save DB queries?

A deep ETag is computed from the entity's version or content hash before loading the full entity. In Spring, `WebRequest.checkNotModified(etag)` compares the ETag against the If-None-Match header early in the request. If they match, the controller can return null immediately — no DB query for the full entity. Implementation: first execute a cheap lightweight query fetching only the version number (e.g., `SELECT version FROM products WHERE id = ?`). Build the ETag string from the version. Call `checkNotModified` — if true, return null (304). Only if the ETag changed do you fetch the full entity. This saves DB load proportional to the cache hit rate.

Q6. How do you generate an ETag — what are the typical strategies?

Common ETag generation strategies: (1) Version number — ETag: `"5"` where 5 is the `@Version` column value; cheap, exact, no hashing needed. (2) Content MD5/SHA hash — MD5 of the serialized response body; more expensive but works for resources without a version column. (3) Last-modified timestamp — ETag derived from the last-modified time of the entity; convenient but has millisecond-precision risks. (4) Composite hash — hash of multiple fields or multiple entities (for collection endpoints). The `@Version` strategy is best for single-entity endpoints because it's $O(1)$ to retrieve, deterministic, and integrates naturally with JPA optimistic locking.

Q7. What does RFC 7232 specify about ETags?

RFC 7232 (HTTP/1.1: Conditional Requests) specifies: (1) ETag syntax — either strong (`"token"`) or weak (`"W/\"token\""`); (2) Strong ETags require byte-for-byte identical responses; weak ETags allow semantically equivalent but byte-different responses (e.g., different gzip encoding). (3) If-Match uses strong comparison — a weak ETag never matches a strong ETag; (4) If-None-Match uses weak comparison — `W/\"abc\"` matches `"abc"` for cache validation; (5) If-Match: `*` means 'resource must exist'; If-None-Match: `*` means 'resource must not exist' (useful for safe PUT-if-not-exists). (6) Multiple ETags can be listed: If-None-Match: `"abc", "xyz"`

Q8. How do ETags integrate with Cache-Control headers?

ETags and Cache-Control work at different layers: Cache-Control (max-age) tells the client how long to use the cached response without contacting the server at all. After max-age expires, the client sends a conditional GET with If-None-Match to check if the resource changed. If unchanged, server returns 304 and the client resets its max-age timer. Typical production setup: Cache-Control: max-age=300, must-revalidate plus ETag: `"v5"`. For the first 5 minutes, the client serves from cache with zero network requests. After 5 minutes, one conditional GET either returns 304 (cheap) or 200 + new data + new ETag. For user-specific resources, use private in Cache-Control to prevent CDN/proxy caching.

Q9. What is the difference between strong and weak ETags?

Strong ETags require that the full response bytes are identical across requests — same content, same encoding, same Content-Type. If the server gzip-compresses some responses but not others (e.g., based on Accept-Encoding), the same resource will have different ETags for compressed and uncompressed variants, breaking conditional requests. Weak ETags (`W/\"token\"`) indicate semantic equivalence — the resource is conceptually the same even if bytes differ (e.g., different encoding). RFC 7232 requires strong comparison for If-Match (write safety) and allows weak comparison for If-None-Match (caching). If you use gzip compression,

generate weak ETags to avoid false mismatches. Spring's `ShallowEtagHeaderFilter` supports `writeWeakETag=true`.

Q10. How does If-Match: * work and when is it useful?

If-Match: * means 'apply the request only if the resource currently exists' (any ETag matches). If the resource does not exist, the server returns 412 Precondition Failed. Conversely, If-None-Match: * means 'apply only if the resource does not exist'. These are useful for: (1) Safe PUT-create — PUT with If-None-Match: * creates a resource only if it doesn't exist, preventing accidental overwrites of existing resources. (2) Safe DELETE — DELETE with If-Match: "etag" ensures you delete exactly the version you last read. If-Match: * on DELETE ensures the resource exists before deleting. These patterns replace the read-then-check-then-write antipattern with atomic HTTP semantics.

Q11. How do you implement conditional requests in Spring without ShallowEtagHeaderFilter?

Use the `WebRequest` API in controller methods. `WebRequest.checkNotModified(String etag)` compares the etag against the If-None-Match header and sets the 304 status if they match, returning true. If the method returns null after this, Spring sends 304. For Last-Modified: `WebRequest.checkNotModified(long lastModifiedTimestamp)`. You can combine both: `checkNotModified(etag, lastModifiedTs)`. Manually set ETag on the response using `ResponseEntity.ok().eTag(etag).body(body)`. The key advantage over `ShallowEtagHeaderFilter` is that you can check the ETag before loading the full resource, saving DB queries.

Q12. What are the risks of using response body hash as an ETag?

Risks of hash-based ETags: (1) Computational cost — hashing the full serialized body for every response is CPU-intensive for large payloads; version-number ETags are O(1). (2) Non-determinism — if the serialized JSON order is non-deterministic (e.g., `HashMap` iteration order), the same entity may produce different hashes on different invocations, defeating caching. (3) Collection instability — a list endpoint's hash changes if any item changes or if order changes; a version-based approach using `MAX(version)` or a separate collection version is more stable. (4) Shallow-filter buffering — the full response must be buffered in memory before hashing, increasing GC pressure.

Q13. How do you handle ETags for collection endpoints (lists)?

Collection ETags are more complex than single-entity ETags. Options: (1) Hash of all items — serialize the full list and hash it; changes if any item changes but expensive. (2) `MAX(version)` — `SELECT MAX(version) FROM products WHERE category=?` gives a cheap, changing value if any product is updated or added. (3) Separate collection version — a dedicated `collection_version` table incremented on any change to collection members. (4) Query count + `MAX(version)` composite. The key challenge is that adding or removing an item must change the collection ETag even if no existing items changed. For large collections, ETags on paginated responses are tricky — consider page-based ETags incorporating page number and page-level version.

Q14. How do ETags compare to using timestamps (Last-Modified) for cache validation?

Last-Modified uses a timestamp (If-Modified-Since header); ETag uses an opaque validator (If-None-Match header). ETags are preferred because: (1) Timestamps have limited precision (HTTP dates have 1-second resolution via RFC 7231); two updates within the same second appear as one change. (2) ETags can represent logical versions regardless of time. (3) For dynamically generated content without a meaningful modification time, ETags are the only option. RFC 7232 says ETags SHOULD be preferred over Last-Modified when both are available. Both can be combined: `WebRequest.checkNotModified(etag, lastModifiedTimestamp)` uses the most recent of the two validations.

Q15. Describe a production incident caused by missing ETags or incorrect ETag implementation.

Classic incident: a high-traffic API returning large JSON payloads without ETags. Every client request fetches the full payload even when data hasn't changed, causing: database overload from redundant queries, high

bandwidth costs, slow client-perceived performance. Fix: add `ShallowEtagHeaderFilter` (immediate bandwidth relief) then migrate hot endpoints to deep ETags (DB query savings). Another incident: using `ShallowEtagHeaderFilter` with gzip compression. The filter runs before gzip encoding in the filter chain, computing ETags from uncompressed bytes. When the client sends `If-None-Match` with the ETag received from a compressed response, the headers don't match (different bytes) → every request returns 200 instead of 304. Fix: disable gzip for ETag-protected endpoints or use weak ETags.

Q16. How does the filter chain order matter for `ShallowEtagHeaderFilter` and gzip?

`ShallowEtagHeaderFilter` must be positioned carefully in the Servlet filter chain. If `GzipFilter` runs BEFORE `ShallowEtagHeaderFilter`, the body is compressed before ETag computation — ETag is based on compressed bytes and is stable across requests with the same `Accept-Encoding`. If `GzipFilter` runs AFTER, the body is uncompressed when `ShallowEtagHeaderFilter` runs — ETag is based on uncompressed bytes, but the client receives compressed bytes — they don't know the original uncompressed ETag, so conditional requests with `If-None-Match` will fail. Best practice: place `ShallowEtagHeaderFilter` AFTER compression, or use weak ETags (`writeWeakETag=true`) to allow semantically equivalent but byte-different representations to match.

Q17. What is the relationship between ETags and optimistic locking?

ETags and `@Version`-based optimistic locking are complementary implementations of the same concept at different protocol layers. `@Version` operates at the database layer: the SQL `WHERE` clause checks the version column. ETag + `If-Match` operates at the HTTP layer: the server checks the `If-Match` header before applying the write. When `@Version` is the ETag source (ETag = version number), the two mechanisms align perfectly: a 412 at the HTTP layer corresponds to a 409 Conflict from the OCC check at the DB layer. They can coexist: `If-Match` provides early rejection without even touching the DB; `@Version` provides a final safety net if `If-Match` is not supplied.

Q18. How do you implement ETag-based PUT-if-create (safe resource creation)?

To create a resource only if it doesn't exist: `PUT /api/resources/{id}` with `If-None-Match: *` header. The server logic: (1) check if resource exists; (2) if exists, return 412 Precondition Failed; (3) if not exists, create and return 201 Created with ETag. In Spring: check `@RequestHeader(value='If-None-Match', required=false)` and if value is `*`, perform the existence check before creating. This is safer than POST because the client controls the ID (idempotent if retried) and the `If-None-Match: *` guard prevents accidental overwrites if the network retries the request. RFC 7232 explicitly supports this pattern.

Q19. How do you test ETag behavior in a Spring Boot integration test?

Use `MockMvc` for full stack testing: (1) `GET /api/resource` → assert 200 and extract `response.getHeader('ETag')`. (2) `GET /api/resource` with `If-None-Match` set to the captured ETag → assert 304. (3) Modify the resource via PUT. (4) `GET` with old `If-None-Match` → assert 200 and new ETag in response. For `If-Match`: (5) PUT with `If-Match` set to old ETag → assert 412. (6) PUT with `If-Match` set to current ETag → assert 200. Use `@SpringBootTest` with `TestRestTemplate` for full container tests, or `MockMvc` for lightweight tests. Verify `ShallowEtagHeaderFilter` is active by checking the ETag header is present.

Q20. How do CDNs interact with ETags, and what should you be aware of?

CDNs (CloudFront, Fastly, Nginx) cache responses and forward conditional requests to origins. Key considerations: (1) The CDN caches both the response body and the ETag header. When the CDN cache expires (max-age), the CDN sends `If-None-Match` to the origin on behalf of the client. If 304, CDN refreshes its cache TTL without downloading the body. (2) For user-specific resources, `Cache-Control: private` prevents CDN caching (only browser cache). (3) Vary: `Accept-Encoding` header tells the CDN to cache separate versions for compressed/uncompressed — important with strong ETags. (4) Some CDNs strip ETag headers

by default — verify CDN configuration preserves ETags for conditional request support. (5) CDN ETags may differ from origin ETags if the CDN modifies the response (e.g., minification) — use weak ETags in this case.